



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240070043
NAME	T. CHANAKYAN
PROJECT TITLE	FLOWER CLASSIFICATION USING K NEAREST NEIGHBORS
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs. M. Divya

Signature of Faculty In-charge

Signature of HoD

Internal Examiner

External Examiner

INTRODUCTION

Machine learning is a branch of artificial intelligence that enables computers to learn patterns from data and make predictions without being explicitly programmed. Classification is one of the most common tasks in machine learning, where the goal is to categorize data into predefined classes. In this project, flower classification is performed using the **K-Nearest Neighbors (KNN)** algorithm. The system predicts the species of a flower based on its physical measurements such as sepal length, sepal width, petal length, and petal width.

The dataset used in this project is the well-known Iris Dataset which contains measurements of three different species of iris flowers. The model learns the relationship between these measurements and the species of the flower. The main objective of this project is to implement the KNN algorithm and classify flowers accurately based on their features.

PROBLEM STATEMENT

Manual identification of flower species requires botanical knowledge and experience. Automating this process using machine learning techniques can improve efficiency and accuracy. The problem addressed in this project is to develop a classification model that can automatically identify the species of a flower using its physical measurements.

GOAL

- Build a classification model using KNN
- Train the model using labeled flower dataset
- Predict flower species accurately
- Evaluate model performance using accuracy, precision, recall, and F1-score

THEORETICAL BACKGROUND

K-Nearest Neighbors is a supervised learning algorithm used for classification. It works by calculating the distance between a test data point and all training data points. The Euclidean distance formula is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The algorithm selects the K nearest neighbors and assigns the class based on majority voting.

Why KNN?

- Simple to implement
- No training phase required
- Works well for small datasets
- Good accuracy for classification problems

ALGORITHM EXPLANATION WITH EXAMPLE

The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning algorithm used for classification problems. It classifies a new data point based on the similarity between the new point and the existing data points in the dataset. In this project, the algorithm uses four features:

- SepalLengthCm
- SepalWidthCm
- PetalLengthCm
- PetalWidthCm

These values are used to determine the nearest neighbors and predict the flower species. The algorithm calculates the distance between the test sample and all training samples. The most common distance metric used is Euclidean distance. Euclidean Distance Formula

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

For multiple features, the formula extends to include all dimensions.

STEPS OF THE ALGORITHM

1. Load the dataset
2. Remove unnecessary columns (Id)
3. Select input features and target variable
4. Split dataset into training and testing sets
5. Choose value of K
6. Calculate distance between test data and training data
7. Sort distances in ascending order
8. Select K nearest neighbors
9. Perform majority voting
10. Assign predicted class

WORKING EXAMPLE USING YOUR DATASET

Consider the following three training samples from your dataset:

SepalLength	SepalWidth	PetalLength	PetalWidth	Species
5.1	3.5	1.4	0.2	Iris-setosa
6.4	3.2	4.5	1.5	Iris-versicolor
6.3	3.3	6.0	2.5	Iris-virginica

Suppose a new flower sample is given:

Test Data:

SepalLength = 5.0

SepalWidth = 3.4

PetalLength = 1.5

PetalWidth = 0.2

Step 1: Calculate Distance

Distance with Sample 1 (Setosa):

$$\sqrt{(5.0 - 5.1)^2 + (3.4 - 3.5)^2 + (1.5 - 1.4)^2 + (0.2 - 0.2)^2}$$

= 0.17 (approx)

Distance with Sample 2 (Versicolor):

= 3

IMPLEMENTATION CODE AND DATASET USED

Dataset Used- [vhttps://www.kaggle.com/datasets/uciml/iris](https://www.kaggle.com/datasets/uciml/iris)

The dataset used in this project is the Iris Dataset. It contains 150 flower samples with four input features and one target variable. The implementation is done using Python and machine learning libraries. First, the required libraries are imported. The pandas library is used to load the dataset. The `train_test_split` function is used to divide the dataset into training and testing data. The `KNeighborsClassifier` is used to implement the K-Nearest Neighbors. The `accuracy_score` function is used to evaluate model performance. The dataset is loaded, the Id column is removed, and the

features and target variable are separated. The model is trained using training data and predictions are made using testing data.

Category	Description / Example
Dataset Name	Iris Dataset
Domain	Machine Learning / Classification
Objective	Classification
Data Source	Public
Number of Samples	150
Data Type	Tabular
Instance Description	One row represents one flower sample
Features (Inputs)	SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm
Feature Types	Numerical
Target Variable	Species
Number of Classes	3(Iris-setosa,Iris-versicolor,Iris-virginica)
License	Open source / Publicly available
Version	Standard sklearn / UCI version

PYTHON CODE

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

import numpy as np

data = pd.read_csv(r"C:\Users\Chanakyan\OneDrive\Documents\Custom Office
Templates\iris.csv")

data = data.drop("Id", axis=1)

X = data.drop("Species", axis=1)

y = data["Species"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, average='macro')

recall = recall_score(y_test, y_pred, average='macro')

f1 = f1_score(y_test, y_pred, average='macro')

print("Accuracy:", accuracy)

print("Precision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)

metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

values = [accuracy, precision, recall, f1]

plt.figure()

plt.bar(metrics, values)
```

```
plt.title("Model Performance Metrics")

plt.xlabel("Metrics")

plt.ylabel("Score")

plt.show()

cm = confusion_matrix(y_test, y_pred)

plt.figure()

plt.imshow(cm)

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.colorbar()

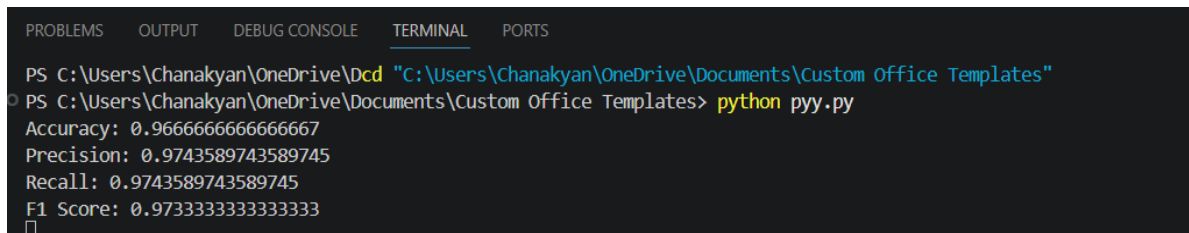
classes = np.unique(y)

plt.xticks(range(len(classes)), classes)

plt.yticks(range(len(classes)), classes)

plt.show()
```

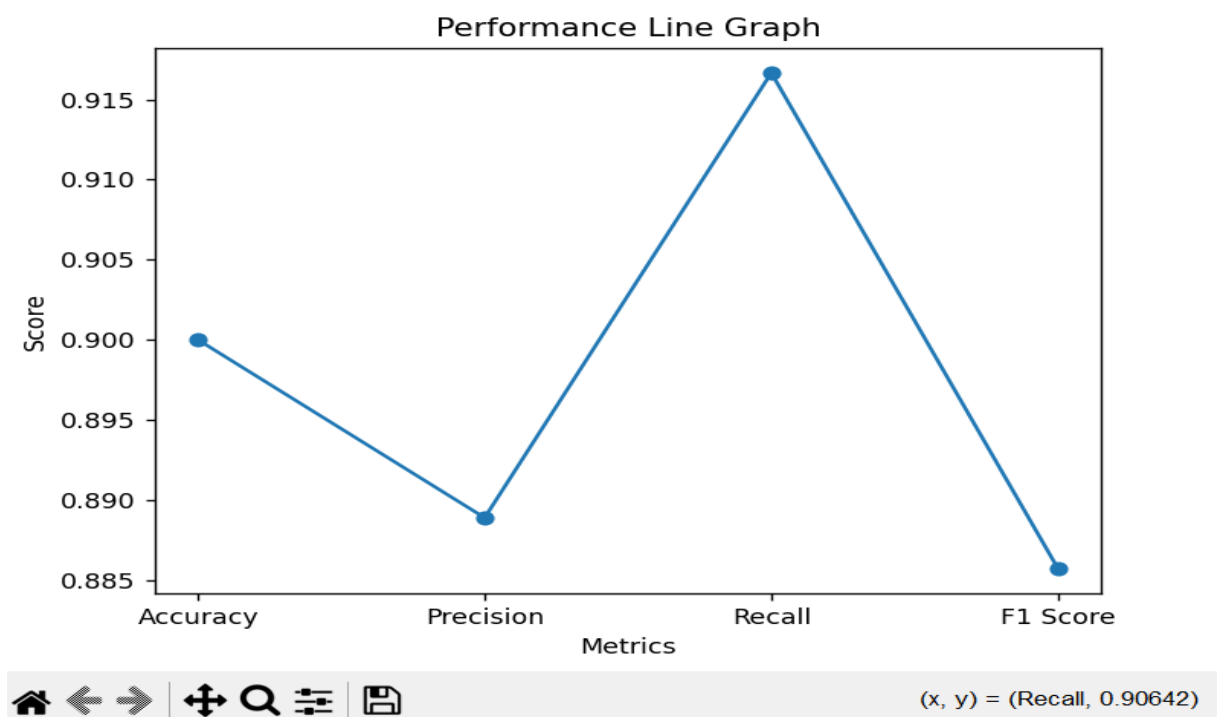
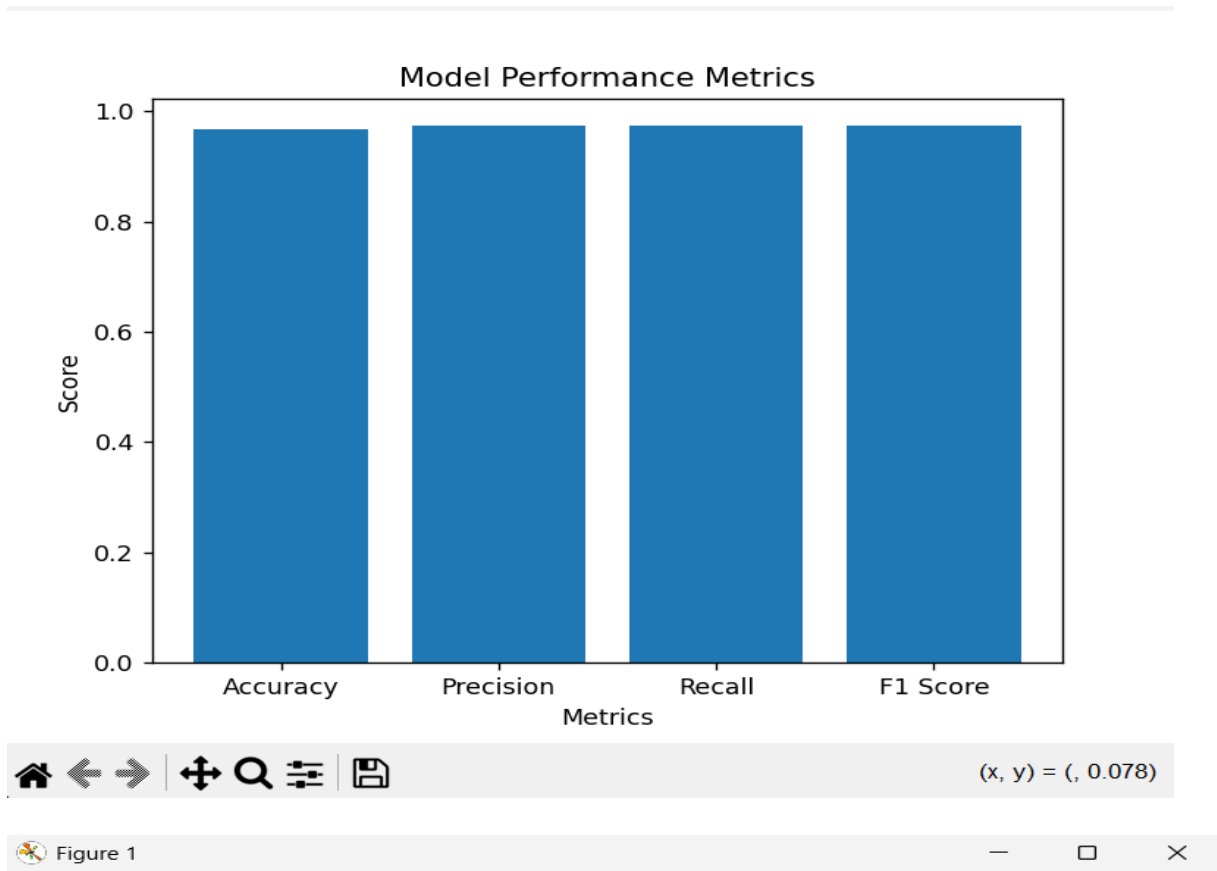
OUTPUT

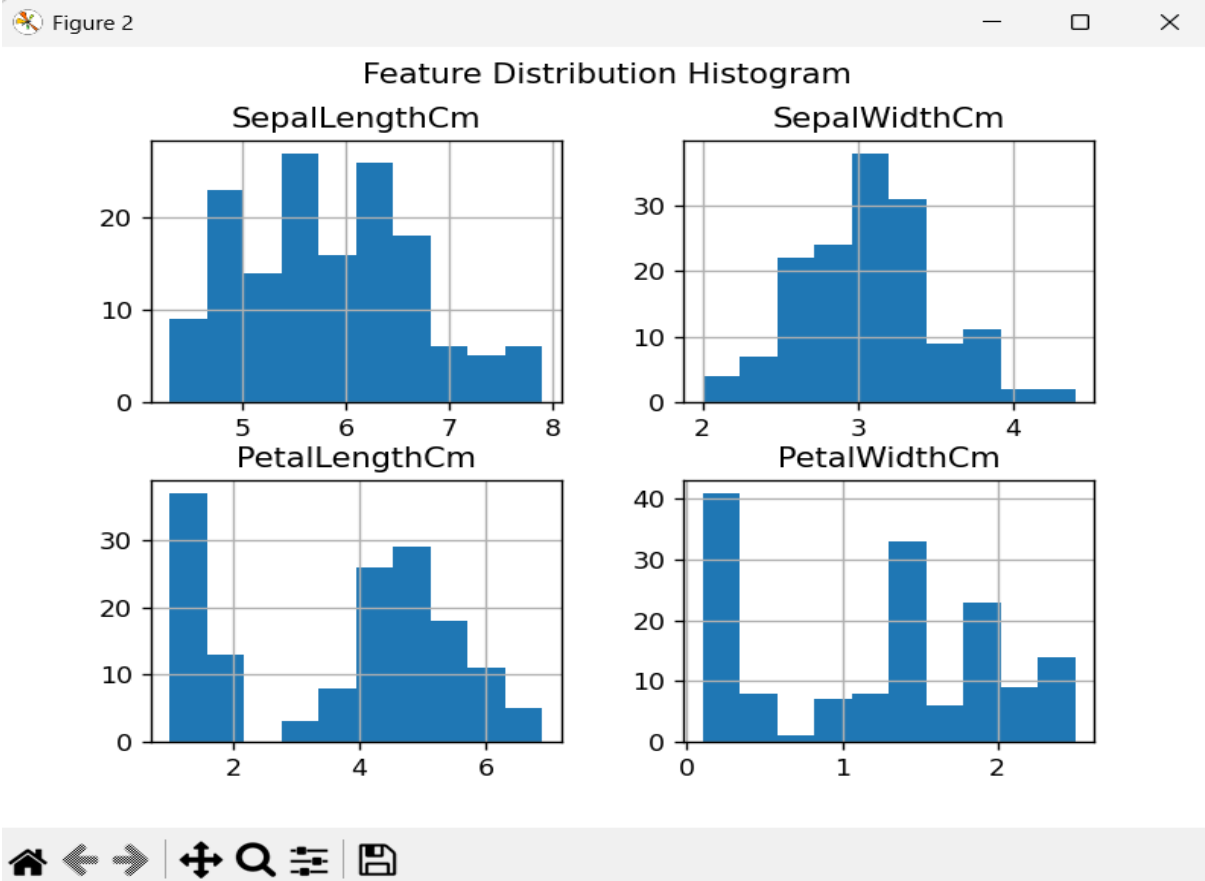


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Chanakyan\OneDrive\Dcd "C:\Users\Chanakyan\OneDrive\Documents\Custom Office Templates"
PS C:\Users\Chanakyan\OneDrive\Documents\Custom Office Templates> python pyy.py
Accuracy: 0.9666666666666667
Precision: 0.9743589743589745
Recall: 0.9743589743589745
F1 Score: 0.9733333333333333
```

FLOWER CLASSIFICATION USING K NEAREST NEIGHBORS





RESULTS AND FUTURE ENHANCEMENT

The K-Nearest Neighbors model achieved high accuracy in classifying flowers using the Iris Dataset. The model was able to correctly distinguish between the three classes—**Iris-setosa**, **Iris-versicolor**, and **Iris-virginica**—based on the input features such as sepal length, sepal width, petal length, and petal width. The performance of the model was evaluated using various metrics including accuracy, precision, recall, and F1 score. The results indicate that the model performs efficiently, with high accuracy and balanced precision and recall across all classes. The confusion matrix also shows that most of the predictions are correct, with very few misclassifications.

This demonstrates that the KNN algorithm is suitable for small and well-structured datasets like the iris dataset. Overall, the model provides reliable classification results and proves that simple machine learning algorithms can achieve good performance when the data is clean and well-defined.

Future Enhancements

Although the current model performs well, several improvements can be made to enhance its performance and usability:

- **Use Larger Dataset:**
The model can be trained on larger and more complex datasets to improve its generalization ability and handle real-world scenarios more effectively.
- **Algorithm Comparison:**
The performance of KNN can be compared with other machine learning algorithms such as Support Vector Machine (SVM) and Random Forest to identify the most suitable model for classification.
- **Parameter Optimization:**
The value of K and distance metrics can be tuned to further improve accuracy and reduce misclassification.
- **Build Web Interface:**
A user-friendly web application can be developed where users can input flower measurements and get predictions instantly.
- **Deploy Mobile Application:**
The model can be integrated into a mobile application for real-time flower classification, making it more accessible and practical.
- **Visualization Enhancements:**
Advanced visualization techniques such as interactive graphs and dashboards can be added for better understanding of the model performance.

Git Hub Link of the project along with the report	https://github.com/Chanakyan233/iris-classification/upload/main
---	---

REFERENCES

- **Scikit-learn Documentation**
<https://scikit-learn.org/stable/documentation.html>
- **Machine Learning by Tom Mitchell (Book)**
<https://www.cs.cmu.edu/~tom/mlbook.html>
- **Iris Dataset – UCI Repository**
<https://www.kaggle.com/datasets/uciml/iris>
- **KNN Algorithm Research Papers**
<https://scholar.google.com/scholar?q=KNN+algorithm+research+papers>
- **Python sklearn Documentation**
<https://scikit-learn.org/stable/>